

Sistemas Operacionais · PUCRS · 2026/II

Contagem de objetos em uma matriz binária

Versão sequencial e versão paralela com processos POSIX

Apresentação em dupla · C89/C90

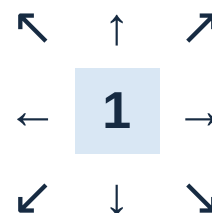
O que conta como um objeto

1	1	0	0	0
1	1	0	0	0
0	0	0	1	0
0	0	0	1	0
1	0	0	0	0

Exemplo 1 do enunciado · 5 × 5

0 é fundo. 1 pertence a um objeto.

As células se conectam por lados **ou diagonais**.



Resultado: 3 objetos

1. Ler a matriz e iniciar a sequencial

src/sequencial.c · linhas 17–26

```
17     if (get_matrix(path, &matrix) != 0) return EXIT_FAILURE;
18     labels = (size_t *)calloc(matrix.cells, sizeof(size_t));
19     if (labels == NULL) {
20         perror("calloc rotulos");
21         free(matrix.values);
22         return EXIT_FAILURE;
23     }
24     result = get_part_objects(&matrix, 0, matrix.rows,
labels, &objects);
25     free(labels);
26     free(matrix.values);
```

5	5			
1	1	0	0	0
1	1	0	0	0
0	0	0	1	0
0	0	0	1	0
1	0	0	0	0

Leitura → rótulos zerados → busca

A entrada é validada antes da contagem.

2. Encontrar um novo componente

src/matrix.c · linhas 112–119

```
112     *objects = 0;
113     for (cell = first; cell < limit; cell++) {
114         if (!matrix->values[cell] || labels[cell - first])
continue;
115         (*objects)++;
116         label = cell + 1;
117         labels[cell - first] = label;
118         top = 0;
119         stack[top++] = cell;
```

1. Pular fundo e células já rotuladas.
2. Somar um objeto.
3. Marcar a célula e colocá-la na pilha.

Rótulo = índice global da célula inicial + 1.

Zero fica reservado para o fundo.

3. Visitar os oito vizinhos

src/matrix.c · linhas 132–135

```
132         next = nr * matrix->cols + nc;  
133         if (matrix->values[next] && !labels[next  
- first]) {  
134             labels[next - first] = label;  
135             stack[top++] = next;
```

Enquanto a pilha tiver células:

1. Retirar a última célula.
2. Verificar os oito vizinhos dentro da faixa.
3. Marcar e empilhar os vizinhos com valor 1.

Marcar **antes** de empilhar evita duplicações.

A pilha fica no heap; a busca não usa recursão.

4. Acompanhar a contagem sequencial

1	1	0	0	0
1	1	0	0	0
0	0	0	14	0
0	0	0	14	0
21	0	0	0	0

Cada rótulo identifica um componente

Busca encerrada: 3 objetos, todos os componentes rotulados.

Objetos: 3

Células na pilha: 0

Simulação da ordem de marcação do código C.

5. Dividir as linhas entre processos

src/paralela.c · linhas 158–162

```
158     base = matrix.rows / processes;
159     extra = matrix.rows % processes;
160     start = 0;
161     for (i = 0; i < processes; i++) {
162         end = start + base + (i < extra);
```

Processo 1 · linhas 1 a 3

Processo 2 · linhas 4 a 6

Exemplo: 6 linhas ÷ 2 processos

Se houver sobra, os primeiros processos recebem uma linha a mais.

Dentro da faixa, a busca é a mesma.

6. Criar os filhos e os pipes

Pai: matriz e divisão

fork()

Filho 1
busca na faixa superior

Filho 2
busca na faixa inferior

pipe 1 ↓

↓ pipe 2

Pai: recebe e consolida

Memória privada nos filhos · comunicação por pipes · encerramento verificado com waitpid

7. Enviar a contagem e as duas bordas

src/paralela.c · linhas 51–56

```
51     result = get_part_objects(matrix, start, end, labels,  
&objects);  
52     if (result == 0 &&  
53         (transfer(fd, &objects, sizeof(objects), 1) != 0 ||  
54         transfer(fd, labels, matrix->cols * sizeof(size_t),  
55         1) != 0 ||  
56         transfer(fd, labels + cells - matrix->cols,  
                    matrix->cols * sizeof(size_t), 1) != 0)) {
```

Cada filho envia:

1. A quantidade de componentes locais.
2. Os rótulos da primeira linha.
3. Os rótulos da última linha.

As células internas já foram resolvidas na busca local.

8. Consolidar os objetos na fronteira

0	0	0	0	0	0	7	7
0	10	10	10	10	0	7	0
0	0	10	10	0	0	0	0
0	0	0	10	10	0	0	0
0	0	0	0	10	0	0	40
41	41	0	0	0	0	40	40

Exemplo 2 · linha laranja separa as faixas

Os demais contatos entre 10 e 28 já têm o mesmo representante. A contagem continua em 4.

Consolidação: $5 - 1 = 4$

Rótulos locais centrais: 10 e 28.

Contato diagonal: linha 3, coluna 3 ↔ linha 4, coluna 4.

9. Evitar a contagem duplicada

src/paralela.c · linhas 89–96

```
89         if (!lower[nc]) continue;
90         a = get_root(parents, upper[col]);
91         b = get_root(parents, lower[nc]);
92         if (a == b) continue;
93         if (rank[a] < rank[b]) { swap = a; a = b; b =
swap; }
94         parents[b] = a;
95         if (rank[a] == rank[b]) rank[a]++;
96         (*objects)--;
```

Union-Find guarda quais rótulos pertencem ao mesmo objeto.

1. Encontrar os representantes.
2. Se já forem iguais, não fazer nada.
3. Se forem diferentes, unir e subtrair um.

O rank orienta a união. `get_root` encurta os caminhos.

10. Executar e comparar as saídas

```
$ ./build/sequencial tests/matrizes/exemplo1.txt  
3  
  
$ ./build/paralela tests/matrizes/exemplo1.txt 2  
3  
  
$ ./build/paralela tests/matrizes/exemplo2.txt 2  
4
```

Exemplo	Esperado	Seq.	Par.
1	3	3	3
2	4	4	4
3	5	5	5
4	6	6	6
5	7	7	7

Cinco matrizes obrigatórias, mesmos resultados.

11. Comparar o desempenho

Matriz 800×800 · cinco repetições · mediana do tempo total



Versão	Tempo	Aceleração
Sequencial	62,26 ms	1,00×
2 processos	56,91 ms	1,09×
4 processos	56,51 ms	1,10×

Aceleração = tempo sequencial ÷ tempo paralelo

Medição local em Linux. Inclui leitura, criação dos processos, pipes e consolidação. A paralela pode ser mais lenta em outros casos.

12. Validação e encerramento

Correção

723 matrizes · 3.813 execuções C

Casos obrigatórios, aleatórios, diagonais e objetos grandes.

Testes também com AddressSanitizer e UndefinedBehaviorSanitizer.

Tratamento de falhas

Entrada inválida e falhas simuladas de memória e fork.

Sem publicar contagem parcial em caso de erro.

A busca local é paralela. A consolidação fica no pai.